

Glossary, Definitions & Pro Tips
Companion to the Platform Engineering Handbook

Gopal Khandale

2026

Contents

How to use this companion	15
Plain English	15
Technical definition	15
Why it matters	15
Gotcha	15
Tip	15
Interview line	15
Section A — Cloud Fundamentals	16
Cloud	16
Plain English	16
Technical definition	16
Why it matters	16
Gotcha	16
Tip	16
Interview line	16
IaaS, PaaS, SaaS	17
Plain English	17
Technical definition	17
Why it matters	17
Gotcha	17
Tip	17
Interview line	17
Region	17
Plain English	17
Technical definition	18
Why it matters	18
Gotcha	18
Tip	18
Interview line	18
Availability Zone (AZ)	18
Plain English	18
Technical definition	18
Why it matters	18
Gotcha	19
Tip	19

Interview line	19
Resource Group	19
Plain English	19
Technical definition	19
Why it matters	19
Gotcha	19
Tip	19
Interview line	20
Section B — Linux & Shell	21
Shell	21
Plain English	21
Technical definition	21
Why it matters	21
Gotcha	21
Tip	21
Interview line	21
Pipe ()	22
Plain English	22
Technical definition	22
Why it matters	22
Gotcha	22
Tip	22
Interview line	22
Background process (&)	22
Plain English	22
Technical definition	22
Why it matters	23
Gotcha	23
Tip	23
Interview line	23
SSH	23
Plain English	23
Technical definition	23
Why it matters	23
Gotcha	23
Tip	24
Interview line	24
SSH LocalForward (port forwarding)	24
Plain English	24
Technical definition	24
Why it matters	24
Gotcha	24
Tip	24
Interview line	25
systemd	25
Plain English	25
Technical definition	25

Why it matters	25
Gotcha	25
Tip	25
Interview line	26
Section C — Git	27
Repository (repo)	27
Plain English	27
Technical definition	27
Why it matters	27
Gotcha	27
Tip	27
Interview line	27
Commit	27
Plain English	27
Technical definition	28
Why it matters	28
Gotcha	28
Tip	28
Interview line	28
Branch	28
Plain English	28
Technical definition	28
Why it matters	28
Gotcha	29
Tip	29
Interview line	29
Pull Request (PR) / Merge Request (MR)	29
Plain English	29
Technical definition	29
Why it matters	29
Gotcha	29
Tip	29
Interview line	30
Rebase vs Merge	30
Plain English	30
Technical definition	30
Why it matters	30
Gotcha	30
Tip	30
Interview line	31
Merge conflict	31
Plain English	31
Technical definition	31
Why it matters	31
Gotcha	31
Tip	31
Interview line	32

Section D — Networking	33
IP Address	33
Plain English	33
Technical definition	33
Why it matters	33
Gotcha	33
Tip	33
Interview line	33
CIDR (Classless Inter-Domain Routing)	34
Plain English	34
Technical definition	34
Why it matters	34
Gotcha	34
Tip	34
Interview line	34
Subnet	34
Plain English	34
Technical definition	34
Why it matters	35
Gotcha	35
Tip	35
Interview line	35
NSG (Network Security Group)	35
Plain English	35
Technical definition	35
Why it matters	35
Gotcha	35
Tip	36
Interview line	36
DNS	36
Plain English	36
Technical definition	36
Why it matters	36
Gotcha	36
Tip	36
Interview line	37
Port	37
Plain English	37
Technical definition	37
Why it matters	37
Gotcha	37
Tip	37
Interview line	37
TLS / HTTPS	38
Plain English	38
Technical definition	38
Why it matters	38
Gotcha	38

Tip	38
Interview line	38
mTLS (mutual TLS)	38
Plain English	38
Technical definition	38
Why it matters	39
Gotcha	39
Tip	39
Interview line	39
Load Balancer	39
Plain English	39
Technical definition	39
Why it matters	39
Gotcha	40
Tip	40
Interview line	40
Section E — Containers	41
Container	41
Plain English	41
Technical definition	41
Why it matters	41
Gotcha	41
Tip	41
Interview line	42
Image	42
Plain English	42
Technical definition	42
Why it matters	42
Gotcha	42
Tip	42
Interview line	42
Dockerfile	42
Plain English	42
Technical definition	43
Why it matters	43
Gotcha	43
Tip	43
Interview line	43
Layer	43
Plain English	43
Technical definition	43
Why it matters	44
Gotcha	44
Tip	44
Interview line	44
Multi-stage build	44
Plain English	44

Technical definition	44
Why it matters	44
Gotcha	44
Tip	45
Interview line	45
Registry	45
Plain English	45
Technical definition	45
Why it matters	45
Gotcha	45
Tip	45
Interview line	46
Image digest	46
Plain English	46
Technical definition	46
Why it matters	46
Gotcha	46
Tip	46
Interview line	46
Section F — Kubernetes	47
Pod	47
Plain English	47
Technical definition	47
Why it matters	47
Gotcha	47
Tip	47
Interview line	48
Node	48
Plain English	48
Technical definition	48
Why it matters	48
Gotcha	48
Tip	48
Interview line	48
Namespace	48
Plain English	48
Technical definition	49
Why it matters	49
Gotcha	49
Tip	49
Interview line	49
Deployment	49
Plain English	49
Technical definition	49
Why it matters	49
Gotcha	49
Tip	50

Interview line	50
ReplicaSet	50
Plain English	50
Technical definition	50
Why it matters	50
Gotcha	50
Tip	50
Interview line	51
StatefulSet	51
Plain English	51
Technical definition	51
Why it matters	51
Gotcha	51
Tip	51
Interview line	51
Service	51
Plain English	51
Technical definition	52
Why it matters	52
Gotcha	52
Tip	52
Interview line	52
ConfigMap	52
Plain English	52
Technical definition	52
Why it matters	52
Gotcha	53
Tip	53
Interview line	53
Secret	53
Plain English	53
Technical definition	53
Why it matters	53
Gotcha	53
Tip	54
Interview line	54
PersistentVolume / PersistentVolumeClaim	54
Plain English	54
Technical definition	54
Why it matters	54
Gotcha	54
Tip	54
Interview line	55
Ingress / Gateway	55
Plain English	55
Technical definition	55
Why it matters	55
Gotcha	55

Tip	55
Interview line	55
ServiceAccount	55
Plain English	55
Technical definition	56
Why it matters	56
Gotcha	56
Tip	56
Interview line	56
RBAC (Role-Based Access Control)	56
Plain English	56
Technical definition	56
Why it matters	56
Gotcha	57
Tip	57
Interview line	57
Controller	57
Plain English	57
Technical definition	57
Why it matters	57
Gotcha	57
Tip	58
Interview line	58
CRD (Custom Resource Definition)	58
Plain English	58
Technical definition	58
Why it matters	58
Gotcha	58
Tip	58
Interview line	58
Operator	59
Plain English	59
Technical definition	59
Examples	59
Why it matters	59
Gotcha	59
Tip	59
Interview line	59
Section G — Terraform	60
Infrastructure as Code (IaC)	60
Plain English	60
Technical definition	60
Why it matters	60
Gotcha	60
Tip	60
Interview line	60
State (Terraform state)	61

Plain English	61
Technical definition	61
Why it matters	61
Gotcha	61
Tip	61
Interview line	61
Remote backend	61
Plain English	61
Technical definition	61
Why it matters	62
Gotcha	62
Tip	62
Interview line	62
Provider	62
Plain English	62
Technical definition	62
Why it matters	62
Gotcha	62
Tip	63
Interview line	63
Module	63
Plain English	63
Technical definition	63
Why it matters	63
Gotcha	63
Tip	63
Interview line	64
State locking	64
Plain English	64
Technical definition	64
Why it matters	64
Gotcha	64
Tip	64
Interview line	64
Data sources	64
Plain English	64
Technical definition	65
Why it matters	65
Tip	65
Interview line	65
Plan vs Apply	65
Plain English	65
Technical definition	65
Why it matters	65
Tip	65
Interview line	66
Drift	66
Plain English	66

Technical definition	66
Why it matters	66
Gotcha	66
Tip	66
Interview line	66
Section H — Helm	67
Helm	67
Plain English	67
Technical definition	67
Why it matters	67
Gotcha	67
Tip	67
Interview line	67
Chart	67
Plain English	67
Technical definition	68
Why it matters	68
Tip	68
Interview line	68
Release	68
Plain English	68
Technical definition	68
Why it matters	68
Tip	68
Interview line	68
Values	69
Plain English	69
Technical definition	69
Why it matters	69
Tip	69
Interview line	69
Umbrella chart	69
Plain English	69
Technical definition	69
Why it matters	69
Gotcha	70
Tip	70
Interview line	70
Section I — Azure	71
Service Principal (SP)	71
Plain English	71
Technical definition	71
Why it matters	71
Gotcha	71
Tip	71
Interview line	71

Managed Identity	72
Plain English	72
Technical definition	72
Why it matters	72
Tip	72
Interview line	72
Workload Identity (AKS)	72
Plain English	72
Technical definition	72
Why it matters	72
Gotcha	72
Tip	73
Interview line	73
AKS (Azure Kubernetes Service)	73
Plain English	73
Technical definition	73
Why it matters	73
Tip	73
Interview line	73
ACR (Azure Container Registry)	73
Plain English	73
Technical definition	74
Why it matters	74
Tip	74
Interview line	74
Log Analytics Workspace (LAW)	74
Plain English	74
Technical definition	74
Why it matters	74
Gotcha	74
Tip	74
Interview line	75
Section J — CI/CD	76
CI (Continuous Integration)	76
Plain English	76
Technical definition	76
Why it matters	76
Tip	76
Interview line	76
CD (Continuous Delivery vs Deployment)	76
Plain English	76
Why it matters	76
Tip	77
Interview line	77
Pipeline	77
Plain English	77
Technical definition	77

Why it matters	77
Tip	77
Interview line	77
Artifact	77
Plain English	77
Technical definition	77
Why it matters	78
Tip	78
Interview line	78
GitOps	78
Plain English	78
Technical definition	78
Why it matters	78
Gotcha	78
Tip	78
Interview line	78
Image promotion (build-once-deploy-many)	79
Plain English	79
Technical definition	79
Why it matters	79
Tip	79
Interview line	79
Section K — Progressive Delivery	80
Blue/Green deployment	80
Plain English	80
Technical definition	80
Why it matters	80
Gotcha	80
Tip	80
Interview line	80
Canary deployment	80
Plain English	80
Technical definition	81
Why it matters	81
Tip	81
Interview line	81
Argo Rollouts	81
Plain English	81
Technical definition	81
Why it matters	81
Tip	81
Interview line	82
AnalysisTemplate	82
Plain English	82
Technical definition	82
Why it matters	82
Tip	82

Interview line	82
Min-traffic gate	82
Plain English	82
Technical definition	82
Why it matters	83
Tip	83
Interview line	83
Section L — Observability	84
Metric	84
Plain English	84
Technical definition	84
Why it matters	84
Tip	84
Interview line	84
Cardinality	84
Plain English	84
Technical definition	85
Why it matters	85
Gotcha	85
Tip	85
Interview line	85
PromQL	85
Plain English	85
Technical definition	85
Why it matters	85
Tip	85
Interview line	86
SLI / SLO / SLA	86
Plain English	86
Technical definition	86
Why it matters	86
Tip	86
Interview line	86
Error budget	87
Plain English	87
Technical definition	87
Why it matters	87
Tip	87
Interview line	87
Reading the Handbook — Pro Tips	88
Common Cross-Cutting Tips	89
When you hit an error	89
When learning a new tool	89
When debugging Kubernetes	89
When writing code/YAML/HCL	89

Career tips	90
End of Glossary	91

How to use this companion

Keep this open alongside the main handbook. When you hit a term in the handbook that's unfamiliar, look it up here.

Each term is broken into six sections:

Plain English

What it actually means, no jargon. Read this first.

Technical definition

The formal definition you'd find in documentation. Read this when you need precision.

Why it matters

When you'd actually use it, and what it solves. Read this to understand the motivation.

Gotcha

The thing that trips people up. Read this to avoid common mistakes.

Tip

Practical advice from experience. Read this for the shortcut.

Interview line

How to drop it in a technical conversation. Read this before interviews.

Section A — Cloud Fundamentals

Cloud

Plain English

Renting computers and services from a big provider (Microsoft, Amazon, Google) instead of buying and racking your own.

You make an API call, get a server in 60 seconds. You pay per hour for as long as you use it.

Technical definition

Pay-per-use access to compute, storage, networking, and managed services exposed via APIs and hosted in geographically distributed data centers. The provider handles hardware, networking, and the lowest layers of software; you handle progressively more depending on which service model you choose.

Why it matters

The cloud lets you treat infrastructure like cattle, not pets. Servers are disposable, re-creatable from code, and scale from one to thousands in minutes. You can build, destroy, and rebuild entire environments to test ideas — something physically impossible with owned hardware.

Gotcha

Cloud costs spiral when nobody's watching. A forgotten VM is \$80/month. A misconfigured Storage Account exporting terabytes can be thousands. Always tag resources, set budgets, and review costs weekly.

Tip

Use cost alerts and budgets from day one. Tag everything with **Project**, **Owner**, **Environment**. When in doubt, prefer managed services over self-managed — the management cost usually outweighs the savings.

Interview line

“The cloud lets us treat infrastructure like cattle, not pets — destroyable and re-creatable from code. Everything is rented and metered, which forces good cost discipline.”

IaaS, PaaS, SaaS

Plain English

Three layers of “how much of the stack the provider runs versus you”:

- **IaaS** — they give you a VM, you do everything from the OS up
- **PaaS** — they run a platform (database, web server), you just deploy your code
- **SaaS** — they run the whole app, you just use it

Technical definition

Service models defined by the **shared responsibility model**:

- **IaaS** (Infrastructure as a Service) — provider manages hardware, virtualization, networking. Customer manages OS, runtime, app. Examples: Azure VM, AWS EC2.
- **PaaS** (Platform as a Service) — provider manages everything up to the runtime. Customer deploys app code + config. Examples: Azure App Service, Heroku, Google App Engine.
- **SaaS** (Software as a Service) — provider runs the entire stack. Customer brings only data and config. Examples: Office 365, Salesforce, GitHub.

Why it matters

Picking the right service level avoids over-engineering. Don’t run a VM if a PaaS will do. Don’t write a Postgres operator if Azure Database for PostgreSQL exists.

Higher layers mean less work and less control. Lower layers mean more work and more flexibility.

Gotcha

Vendor lock-in increases as you go up the stack. SaaS is easy to start, painful to leave. PaaS often uses proprietary APIs. IaaS is the most portable but the most work.

Tip

When choosing, ask: “What’s the smallest layer I can use to get what I need?” Start there. Move down (toward IaaS) only when the higher layer can’t deliver.

Interview line

“We picked AKS — managed Kubernetes, a PaaS — instead of self-managed Kubernetes on VMs because we want the cloud handling control plane upgrades and certificate rotation, not us. That tradeoff was right for our scale.”

Region

Plain English

A geographic area with cloud data centers. Examples: East US 2, West Europe, Central India.

You pick a region close to your users so they get low latency.

Technical definition

A set of data centers within the same geographic area, low-latency networked together, typically containing 3 Availability Zones. Each region is independent — failures in one region don't affect others.

Why it matters

Four reasons to pick a specific region:

1. **Latency** — closer to users = faster requests
2. **Data residency** — GDPR, India's DPDP Act, etc., require data stays in country
3. **Service availability** — not every service is in every region (GPU SKUs, preview features)
4. **Disaster recovery** — pair regions across geographic separation

Gotcha

Costs vary by region. `eastus` is usually cheapest in the US, `centralindia` is sometimes 20% more, and some regions don't support every service tier (e.g., Free Trial restrictions on B-series VMs).

Tip

Always confirm your chosen region has the SKUs you need before committing. Use `az vm list-skus --location <region>` to verify.

Interview line

“We picked `eastus2` because it has the latest SKUs and broad service coverage. For HA we'd pair it with `westus2` — that's a recognized paired-region by Azure with cross-region replication built in for some services.”

Availability Zone (AZ)

Plain English

One physically independent data center within a region. Has its own power, cooling, and network. If one AZ burns down, the other AZs in that region keep running.

Technical definition

A physically isolated location with independent power, cooling, and network infrastructure, within a region. Typically 3 AZs per region. AZs are connected by low-latency dedicated networking.

Why it matters

A single data center can fail (fire, power outage, network outage). If all your nodes are in one AZ, you go down. Spread across AZs for high availability.

In Kubernetes, this is done with `topologySpreadConstraints` or node pool zone settings.

Gotcha

Cross-AZ traffic costs more than same-AZ traffic. For chatty microservices, this can add up. Some workloads (batch jobs) might be fine in one AZ; user-facing services need multi-AZ.

Tip

Look at your network bill, not just compute. Cross-AZ chatty traffic is a common cost surprise.

Interview line

“We spread our AKS node pool across three AZs so a data center failure doesn’t take us out. Cross-AZ pod traffic costs slightly more, but the reliability gain pays for it. Same-AZ would save ~10-15% on intra-cluster traffic costs but expose us to single-DC failure.”

Resource Group

Plain English

A folder for related Azure resources. Everything in a resource group can be deleted with one command.

Technical definition

A logical Azure container whose lifecycle is managed together. RBAC roles and tags can apply at this level. Resources can only be in one resource group, but resources from different RGs can reference each other.

Why it matters

`az group delete -n myrg --yes` cleans up everything inside in one call. This is critical for ephemeral environments (per-PR review apps, sandboxes). Tags applied at RG level cascade for cost reporting.

Gotcha

You can’t easily nest resource groups. They’re flat. To organize hierarchically, use Management Groups (at the subscription/billing level).

Moving resources between RGs is supported but tricky — locks, dependencies, role assignments can break.

Tip

Use one resource group per environment-app combination — `myapp-rg-dev`, `myapp-rg-prod`. Makes teardown trivial and cost reporting clean.

Interview line

“We use one RG per environment-app combination — `gskplat-rg-platform` for our shared AKS cluster, separate RGs for ephemeral test environments. Makes teardown trivial: `az group delete` removes everything. Tags at the RG level cascade to all resources for cost allocation.”

Section B — Linux & Shell

Shell

Plain English

The thing that reads what you type and runs it. Bash, zsh, fish — they’re all shells. The black/green text terminal you see is the shell.

Technical definition

A command interpreter that provides a user interface to the operating system. Reads commands (interactive or from a script), parses them, executes them, and displays output. Common Unix shells: bash, zsh, fish, dash. Windows: PowerShell, cmd.

Why it matters

You’ll spend hours every day in a shell. Mastering it 5x’s your productivity. Pipes, redirection, scripting — these are how Unix philosophy becomes real work.

Gotcha

bash and zsh have subtle differences. Test conditions: `[ $x -eq 5 ]` works in both, `[[ $x -eq 5 ]]` is bash/zsh extension. `(( ))` for arithmetic is bash/zsh. Plain `sh` (Bourne shell, often dash on Debian) doesn’t support these.

Tip

Learn one shell deeply (bash for portability across systems, zsh for daily interactive use with plugins). Memorize 10 patterns, copy-paste the rest from a personal notes file.

Interview line

“I’m fluent in bash for scripting and zsh for daily use. I keep a personal cheatsheet of common patterns — text processing pipelines, find + xargs, awk one-liners — that I copy-paste rather than re-derive each time.”

Pipe (|)

Plain English

Takes the output of one command and feeds it as the input of the next. Lets you chain simple commands into powerful workflows.

Technical definition

A Unix construct that connects stdout of one process to stdin of another via a kernel pipe (a buffered, in-memory FIFO). Multiple commands chained with pipes form a pipeline — all processes run concurrently, with the kernel managing flow control.

Why it matters

The Unix philosophy: small tools that do one thing well, composed via pipes. `ps aux | grep node | awk '{print $2}'` lists processes, filters for “node”, extracts PIDs — three small tools, one clear job.

Gotcha

Pipes are stream-oriented. They start everything in parallel. The second command can start processing the first command’s output before the first command finishes. This matters for slow producers and fast consumers — your `tail -f` and `head -5` combinations don’t always behave as expected.

Tip

For commands that need to know when input is complete (`sort`, `uniq`), the upstream command must close stdout. Use `< <(cmd)` or temp files when pipe semantics don’t work.

Interview line

“Pipes are concurrent — commands in a pipeline run in parallel. I’d reach for `xargs` or `parallel` for explicit fan-out when ordering matters.”

Background process (&)

Plain English

Run something but don’t wait for it. The shell returns immediately, the process keeps running.

Technical definition

When you append `&` to a command, the shell forks the process and disowns its stdin, but keeps the process attached to the shell session. Closing the terminal sends a `SIGHUP`, which usually kills the background job.

Why it matters

Long-running commands (port-forwards, dev servers, build watches) need to keep running while you do other things in the same shell.

Gotcha

Background jobs die when the parent shell exits. Use `nohup`, `disown`, or `screen`/`tmux` to keep them running across logouts.

Tip

```
nohup mycommand > /tmp/mycommand.log 2>&1 &
```

This pattern: `nohup` ignores `SIGHUP`, redirects all output to a log file, runs in background. Process survives logout.

For complex sessions, use `tmux` — split panes, detach/reattach, much better than `nohup`.

Interview line

“For long-running CI test loops or port-forwards, I prefer `tmux` over `nohup`. `nohup` is fine for one-off scripts. For anything I’d reattach to later, `tmux` gives me a persistent session with full history.”

SSH

Plain English

A secure way to log into a remote computer. Encrypted, key-based, the standard for server access.

Technical definition

Secure Shell — an encrypted protocol for remote access (interactive shell), file transfer (`scp`, `sftp`), and TCP tunneling. Uses asymmetric cryptography for authentication and key exchange, symmetric for bulk data.

Why it matters

Every server, every cloud VM, every Linux/Mac machine — you reach them via SSH. SSH keys + `ssh-agent` is the standard authentication workflow.

Gotcha

Password-based SSH is widely brute-forced on the internet. Always use SSH keys (preferably Ed25519). Disable password auth on internet-facing servers.

Permissions on `~/.ssh/` and `~/.ssh/authorized_keys` matter — too open and SSH refuses to use them. `chmod 700 ~/.ssh` and `chmod 600 ~/.ssh/authorized_keys`.

Tip

Keep an `~/.ssh/config` file with aliases for every server you touch:

```
Host webserver
  HostName web1.example.com
  User deploy
  Port 22022
  IdentityFile ~/.ssh/web_ed25519
```

Then `ssh webserver` instead of remembering `host/port/user/key` every time.

Interview line

“I use SSH keys with `ssh-agent` and `ssh-add`. For production access I’d push for certificate-based SSH (Teleport, Smallstep) with short-lived certs over shared SSH keys — easier audit, easier revocation.”

SSH LocalForward (port forwarding)

Plain English

Make a service running on a remote server show up as if it’s on your local machine. Useful for accessing things behind a firewall.

Technical definition

Tunnels TCP traffic from `localhost:<port>` on your machine through an SSH connection to `<remote-host>:<port>` accessible from the SSH server. The remote port is reached as the SSH server sees it.

Why it matters

Lets you access internal services (Jenkins on a private network, Postgres in a VPC) without exposing them publicly. Critical for working with cloud resources from a laptop.

Gotcha

The forwarded service usually binds to `127.0.0.1` on the remote server — that’s fine, the tunnel target is “localhost from the server’s perspective”, not “localhost on your laptop.”

If you can’t connect, check:

1. The service is actually running on the remote server (`ss -tln | grep <port>`)
2. The SSH config syntax is right
3. Local firewall isn’t blocking the local port

Tip

Add `LocalForward` lines to your `~/.ssh/config`:

```
Host azurelab
  HostName 172.30.44.145
  User aziro
  LocalForward 53000 localhost:3000
  LocalForward 53100 localhost:3100
```

Then `ssh azurelab` automatically opens all the tunnels. No more long command lines.

Interview line

“For developer access to internal services, I prefer SSH port forwarding over VPN — narrower scope, easier auditing, doesn’t need a corporate VPN client. Production access goes through a bastion or Teleport for full audit logging.”

systemd

Plain English

The boss process that starts and watches all the services on a modern Linux system. PID 1. The thing that runs everything else.

Technical definition

An init system and service manager for Linux. Replaces older init systems (SysV init, Upstart). Manages services, mounts, sockets, timers, and provides journal-based logging via `journalctl`.

Why it matters

All your long-running daemons (nginx, postgres, docker, kubelet) live under systemd. Diagnose with `systemctl status`, follow logs with `journalctl -fu`.

Gotcha

Containers don’t need systemd. The container runtime IS the supervisor. Running systemd inside a container is rare, awkward, and usually wrong.

systemd unit file syntax is its own language. Mistakes in `[Service]` section paths cause confusing failures — read the journal carefully.

Tip

```
sudo systemctl status nginx
sudo systemctl restart nginx
sudo systemctl enable nginx      # start at boot
journalctl -u nginx --since "1 hour ago"
journalctl -fu nginx             # follow logs
```

These five commands cover 95% of daily use.

Interview line

“For VMs I’d write systemd unit files for any long-running service — handles restart, logging, dependencies. For containers, the container runtime handles supervision, so systemd is unnecessary inside the container.”

Section C — Git

Repository (repo)

Plain English

A project's folder with version history attached. Lives in `.git/` inside the project directory.

Technical definition

A directory containing a `.git/` subdirectory that holds the entire history of changes as a graph of commits. Each commit is a snapshot of files plus metadata (parent, author, timestamp, message).

Why it matters

Version control is non-negotiable for any non-trivial work. You can experiment freely, revert mistakes, and collaborate without overwriting each other's work.

Gotcha

`.git/` is fragile. Don't manually edit files inside it. Use git commands.

Cloning over slow networks is slow because git transfers history. Use `--depth=1` for CI clones.

Tip

Always start projects with `git init` or `git clone`. Even for “just experimenting.” You'll thank yourself later when you need to revert a 4 AM change.

Interview line

“I git-init every directory I work in, even throwaway ones. The cost is nothing, the benefit is being able to bisect what broke when something goes sideways at 2 AM.”

Commit

Plain English

A snapshot of your project at a moment in time, with a message describing what changed.

Technical definition

A git object containing:

- A tree (filesystem state)
- Author and committer info
- Parent commit reference(s)
- A message

Identified by a SHA-1 hash (40 chars, often abbreviated to 7).

Why it matters

Smaller commits = easier to review, easier to revert, better blame info. A clean history is a debugging tool.

Gotcha

`git commit -am` skips new files (only stages tracked-modified). Use `git add` explicitly to be sure. `--amend` rewrites the previous commit. Never amend a commit that's been pushed to a shared branch.

Tip

Write commit messages in imperative mood: “Add X” not “Added X”. Industry convention. The convention reads like a directive to the codebase.

Subject line 50 chars. Blank line. Then a body explaining WHY (not WHAT — the diff shows what).

Interview line

“We tag images by git short SHA. Every deploy is traceable to a specific commit. Rollback is `git revert <commit>` and ArgoCD reconciles. Audit trail comes for free.”

Branch

Plain English

A line of work, like a parallel timeline. You can switch between branches to work on different things.

Technical definition

A movable pointer to a commit. The “current branch” is what HEAD points to. New commits advance the branch pointer.

Why it matters

Branches let you experiment, work on features in isolation, and keep main always-deployable.

Gotcha

`git checkout` does two different things: switch branches AND restore files. Newer git versions split this into `git switch` (branches) and `git restore` (files). Use the new commands when you can — clearer intent.

Tip

Keep branches short-lived. Long branches diverge from main and become painful to merge. Rule: open the PR within 3 days of starting the branch.

Interview line

“I keep feature branches short — a day, maybe two. Long-running branches end up in merge hell. If a feature needs more than a week, I’d break it into smaller PRs that can land independently behind a feature flag.”

Pull Request (PR) / Merge Request (MR)

Plain English

“Please merge my branch into the main branch.” A GitHub/GitLab concept, not Git itself.

Technical definition

A web-UI workflow for proposing changes from one branch to another. Combines:

- Diff view
- Discussion thread (comments on specific lines)
- CI integration (status checks)
- Review approvals
- Merge controls

Why it matters

This is THE gate between work and production. Code review catches bugs. Automated tests catch regressions. The audit trail is a compliance artifact.

Gotcha

Big PRs get bad reviews. Reviewers can’t spot real issues in 1000 lines, so they nitpick formatting. Keep PRs 400 lines of real code.

Tip

Open PRs early as draft. Use the description for what + why + how to test:

```
## Summary
What this PR does.
```

Why

The reason / linked ticket.

How to test

Steps to verify.

Risks

What might go wrong.

Reviewers love a well-written description.

Interview line

“I keep PRs small — 300-400 lines of real code at most. Big PRs get bad reviews because reviewers can’t actually evaluate them. If a feature requires a big PR, I’d refactor it into a stack of small PRs that each land independently behind a feature flag.”

Rebase vs Merge

Plain English

Two ways to combine branches. **Merge** keeps history showing what really happened. **Rebase** rewrites history to make it linear and clean.

Technical definition

- `git merge` creates a new “merge commit” with two parent commits. History is a graph.
- `git rebase` replays your commits on top of the target branch, creating new commits (different SHAs). History is linear.

Why it matters

Rebased history is easier to read but loses information about when branches diverged. Merge history shows the actual development pattern.

Teams pick one convention and stick to it.

Gotcha

Never rebase shared branches. You rewrite history that others have based on. They’ll have to do painful resets. Rebase only your local branches before pushing.

Tip

Use `git pull --rebase origin master` to keep your local work on top of the latest master without merge commits. Set it globally:

```
git config --global pull.rebase true
```

Interview line

“Local branches: rebase to keep history clean. Shared branches: merge to preserve the development story. The rule is ‘never rewrite history that’s been pushed.’ Teams that mix this get conflicts and bad blame info.”

Merge conflict

Plain English

Two changes on the same line — git can’t decide. You decide.

Technical definition

When the 3-way merge algorithm can’t auto-resolve overlapping changes between two branches. Git puts conflict markers in the affected files and pauses, waiting for resolution.

Why it matters

You’ll hit these constantly when multiple people work on the same codebase. Knowing how to resolve them fast is core skill.

Gotcha

Conflict markers look ugly:

```
<<<<<< HEAD
your version
=====
their version
>>>>>> commit-sha
```

Don’t just delete markers — actually decide which version (or a blend) is correct.

Tip

```
# See files with conflicts
git diff --name-only --diff-filter=U

# After resolving each file
git add <file>

# Continue the merge/rebase
git rebase --continue
# or
git merge --continue
```

For complex conflicts, use a visual merge tool: `git mergetool`. VS Code has a built-in 3-way merge view.

Interview line

“Conflicts are usually a sign of two parallel changes to the same area. Fixing the conflict is the easy part — the harder question is whether the changes are actually compatible. I’d verify with both authors before merging if it’s not obvious.”

Section D — Networking

IP Address

Plain English

A computer's mailing address on a network. Like a phone number, but for computers.

Technical definition

A 32-bit (IPv4) or 128-bit (IPv6) number identifying a network interface. IPv4 is usually written as four octets: 10.0.1.15. IPv6 uses eight hex groups separated by colons.

Why it matters

Everything network-related ultimately resolves to IP addresses. DNS exists to map human-friendly names to IPs.

Gotcha

A single host can have many IPs — one per network interface, multiple per interface (aliases). When you ping a hostname, the actual IP depends on DNS resolution at that moment.

Tip

Memorize the private IP ranges (RFC 1918):

- 10.0.0.0/8 — used by most cloud VNets
- 172.16.0.0/12 — Docker default
- 192.168.0.0/16 — home routers

If you see one of these, it's a private network. Public IPs are anything else.

Interview line

“Cloud VNets typically use 10.0.0.0/16 or similar from RFC 1918. Subnets are slices of that — 10.0.1.0/24 for the AKS pool, 10.0.3.0/27 for the bastion subnet.”

CIDR (Classless Inter-Domain Routing)

Plain English

A compact way to write “this range of IP addresses.” /24 means 256 addresses. /16 means 65,536. The smaller the number after the slash, the bigger the range.

Technical definition

Notation specifying an IP address followed by /N, where N is the count of network-prefix bits. 10.0.0.0/24 means the first 24 bits are fixed, leaving 8 bits (256 combinations) free for hosts.

Why it matters

Cloud VNets and subnets are sized in CIDR. Get this wrong and you can’t fit your pods. AKS nodes need subnet IPs; Azure CNI pods need many more.

Gotcha

Subnets need overhead. A /24 has 256 IPs, but Azure reserves 5 for routing/DHCP. Effective usable: 251. Plan for 10-20% growth.

Tip

Memorize sizes:

/8	= 16,777,216 IPs	(massive)
/16	= 65,536	(typical VNet)
/24	= 256	(typical subnet)
/27	= 32	(small subnet - bastion, mgmt)
/30	= 4	(point-to-point links)
/32	= 1	(single host)

Interview line

“For our AKS subnet I sized /24 — gives us 251 usable IPs, enough for 100+ kubernetes pods. Production with Azure CNI overlay or full CNI would need much larger subnets — one IP per pod, not just per node.”

Subnet

Plain English

A slice of a larger network (VNet). Lets you separate things logically — one subnet for the cluster, one for management, one for future apps.

Technical definition

A range of IPs within a VNet, often with its own routing, NSG, and route table. Subnets are CIDR-defined and live in a single Azure region/VNet.

Why it matters

Logical separation. Different subnets can have different firewall rules (NSGs), routing (UDRs), and access patterns. Lets you implement zone-based security.

Gotcha

Once a subnet is in use, resizing it is hard. Plan generously upfront. Removing a CIDR range from a subnet requires moving all resources off it first.

Tip

Document subnet purposes clearly in Terraform comments:

```
# snet-aks: Kubernetes node pool (10.0.1.0/24)
# snet-mgmt: bastion/jumpbox (10.0.3.0/27)
# snet-apps: reserved for future PaaS apps (10.0.2.0/24)
```

Interview line

“We use three subnets — AKS nodes, management, future apps. Each has its own NSG, so the blast radius of a misconfiguration stays small. The mgmt subnet has an explicit allow for SSH from my home IP only.”

NSG (Network Security Group)

Plain English

A stateful firewall attached to subnets or VMs. Defines what traffic is allowed in or out.

Technical definition

An Azure resource holding a list of allow/deny rules with priorities (lower number = higher priority). NSGs are stateful — return traffic is auto-allowed. Applied to NICs or subnets.

Why it matters

Without NSGs, anything in your VNet can talk to anything else. NSGs let you implement zone-based security: web tier → app tier → DB tier, with explicit allows.

Gotcha

High-priority Deny rules shadow lower-priority Allow rules. This includes Azure’s IMPLICIT Allow rules at priority 65000+.

Our lab broke in iter 4 because of this. Pod-to-pod traffic across nodes stopped working when we added a Deny-all at priority 4000. The fix: add explicit `AllowVnetInbound` (priority 1000) and `AllowAzureLoadBalancerInbound` (priority 1100) BEFORE the deny.

Tip

If you write an explicit “DenyAll” rule, you **MUST** think about what implicit allows you’re shadowing. Document the explicit allows alongside the deny:

```
# These three rules MUST stay together. The Deny shadows Azure implicit allows.
security_rule { name = "AllowVnetInbound"          priority = 1000 ... }
security_rule { name = "AllowAzureLoadBalancerIn"  priority = 1100 ... }
security_rule { name = "DenyAllInboundExplicit"    priority = 4000 ... }
```

Interview line

“NSGs are stateful, priority-ordered. The gotcha is that an explicit Deny at high priority shadows Azure’s implicit Allows. You have to re-add explicit `AllowVnetInbound` and `AllowAzureLoadBalancerInbound` if you’re using a default-deny pattern. We hit this when scaling the cluster — pod-to-pod across nodes broke until we added the explicit allows.”

DNS

Plain English

The phone book of the internet. Maps human-friendly names to IP addresses.

Technical definition

Domain Name System — a distributed hierarchical naming system. Resolvers query authoritative servers for various record types: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), TXT (text/verification), NS (nameserver).

Why it matters

You don’t remember `20.94.18.66` — you remember `myapp.example.com`. DNS makes the internet usable.

In Kubernetes, CoreDNS resolves cluster-internal names like `backend.dev.svc.cluster.local`.

Gotcha

DNS caching delays propagation. After changing a record, browsers and OS resolvers may use stale results for minutes to hours. Set TTLs intentionally — low (60s) for things that change, high (1 day) for stable records.

Tip

```
dig myapp.example.com          # query a domain
dig +trace myapp.example.com   # see resolution path
nslookup myapp.example.com     # simpler tool
```

For Kubernetes internal DNS troubleshooting:

```
kubectl run dnsdebug --rm -it --image=busybox:1.36 -- nslookup backend.dev.svc.cluster.local
```

Interview line

“In Kubernetes, CoreDNS handles cluster DNS. Services are resolvable at `<svc>.<ns>.svc.cluster.local`. Pods have a search path so within the same namespace you can use just `<svc>`. Cross-namespace requires the namespace prefix.”

Port

Plain English

A door number on a computer. Different apps listen on different ports. Web servers: 80 or 443. SSH: 22. Postgres: 5432.

Technical definition

A 16-bit number (0-65535) identifying a specific endpoint on an IP address for TCP or UDP traffic. The combination of IP + port uniquely identifies a network endpoint.

Why it matters

Lets multiple services run on one IP. The OS routes incoming packets to the right process based on the destination port.

Gotcha

Ports below 1024 require root to bind. Always use 1024 for user-mode apps. Setting `CAP_NET_BIND_SERVICE` on a binary allows it to bind low ports without full root.

Tip

Memorize common ports:

22	SSH
80	HTTP
443	HTTPS
3306	MySQL
5432	PostgreSQL
6379	Redis
9090	Prometheus
6443	Kubernetes API

Interview line

“For our backend we used port 5678 — non-standard, above 1024 so no root needed, doesn’t conflict with anything obvious. The container’s `EXPOSE 5678` is documentation; the actual port-opening happens at Service definition time in Kubernetes.”

TLS / HTTPS

Plain English

Encrypted communication over the internet. The padlock icon in the browser. Means the connection is encrypted and the server is who it claims to be.

Technical definition

Transport Layer Security — protocol providing confidentiality (encryption), integrity (no tampering), and authentication (server’s identity verified via X.509 certificate). Uses asymmetric crypto for the handshake, symmetric for bulk data. HTTPS is HTTP over TLS.

Why it matters

All public-facing traffic should be HTTPS. Even internal services benefit from TLS to prevent passive eavesdropping by attackers who’ve breached the network.

Gotcha

TLS certificates expire. Forgetting to renew breaks the site. Always automate renewal (cert-manager + Let’s Encrypt, ACM, Azure Key Vault certs).

Tip

Check a cert’s expiry from the command line:

```
echo | openssl s_client -connect myapp.example.com:443 2>/dev/null | openssl x509 -noout -dates
```

Interview line

“For public traffic I’d use cert-manager with Let’s Encrypt for automated renewal. For service-to-service inside the mesh, Istio handles mTLS automatically — no certs to manage, the control plane rotates them every 24 hours.”

mTLS (mutual TLS)

Plain English

TLS where BOTH sides prove who they are. Server and client both present certificates. Standard for service-to-service auth in zero-trust networks.

Technical definition

Bidirectional certificate validation extending normal TLS. The TLS server (in addition to presenting its cert) requires the client to present a valid cert. Both sides verify each other against trusted CAs.

Why it matters

Service mesh (Istio, Linkerd) uses mTLS so services authenticate each other, not just users. This is zero-trust networking: never trust IP-based access; always verify cryptographically.

Gotcha

Istio “PERMISSIVE” mTLS accepts both plaintext and mTLS during rollout. “STRICT” rejects plaintext. Going from PERMISSIVE to STRICT before all services have sidecars breaks the non-sidecar ones.

Roll out mTLS in two phases: deploy sidecars everywhere (PERMISSIVE), then flip to STRICT once everything has sidecars.

Tip

Verify mTLS is actually working:

```
istioctl authn tls-check <pod>.<ns>
```

This tells you whether traffic to a pod is mTLS or plaintext.

Interview line

“Our lab uses Istio PERMISSIVE mTLS — sidecars accept both plaintext and mTLS. Production would be STRICT once all services have sidecars. The migration path is: deploy sidecars everywhere first, then flip the mesh-wide PeerAuthentication to STRICT.”

Load Balancer

Plain English

Distributes incoming traffic across multiple backend servers. If you have 3 web servers, the LB sends each request to one of them (round-robin, least-connections, etc.).

Technical definition

A device or service that distributes incoming network connections across multiple backend instances. Operates at:

- **L4** (TCP/UDP) — Azure Load Balancer, AWS NLB
- **L7** (HTTP) — Azure Application Gateway, AWS ALB, Istio ingress gateway

Why it matters

Single point of entry for users. Automatic failover when an instance dies. Spread load to scale horizontally.

Gotcha

L4 LBs don't understand HTTP — they can't route by URL path or header. Use L7 (or service mesh) when you need URL-based routing or HTTP-aware features (sticky sessions, TLS termination, header rewrites).

Tip

In Kubernetes, `type: LoadBalancer` Service creates a cloud LB pointing at your nodes. You'll usually want one global LB (Istio ingress, Nginx ingress) and route to Services via Ingress/VirtualService — not one cloud LB per Service.

Interview line

“For external traffic I use one cloud LB pointing at the Istio ingress gateway. From there, VirtualServices route by Host header and path. One LB IP, many services behind it. Cheaper and easier to manage than one LB per Service.”

Section E — Containers

Container

Plain English

A self-contained box with your app inside. Drop it on any Linux machine and it runs the same way. Like a shipping container — handle uniformly, contents portable.

Technical definition

A process or group of processes isolated by Linux kernel features:

- **Namespaces** isolate what processes can see (PID, network, mount, user, IPC, UTS)
- **Cgroups** limit CPU, memory, I/O usage
- **Union filesystems** (overlayFS) layer filesystem changes efficiently

A container shares the host kernel but has its own filesystem view, network stack, process tree.

Why it matters

Containers solve “works on my machine.” Same image runs identically on developer laptop, CI runner, production cluster. Build/test/deploy becomes deterministic.

Compared to VMs: containers boot in seconds, are megabytes in size, minimal overhead.

Gotcha

A container is NOT a tiny VM. There’s no separate kernel. If the host kernel has a privilege escalation vulnerability, every container on it is potentially exposed.

This is why you patch hosts aggressively and run containers as non-root.

Tip

Keep images small. Use multi-stage builds + minimal base images:

- **alpine** — ~5 MB
- **distroless** — Google’s runtime-only images
- **scratch** — empty base, only your statically-compiled binary

Interview line

“Containers package an app with its runtime so it’s portable. They’re isolated via Linux kernel features — namespaces for what they can see, cgroups for what they can use — but they share the host kernel, which is the key distinction from VMs.”

Image

Plain English

A blueprint for a container. The “frozen” state. You can have many running containers from one image.

Technical definition

An ordered set of read-only filesystem layers plus metadata (config, entrypoint, env vars), identified by a SHA-256 digest. Stored in registries; pulled to nodes when needed.

Why it matters

Images are the deployable unit. Build once, deploy many times. Versioned, immutable (when properly tagged), portable across registries.

Gotcha

Images can be huge. A Node.js image with dev deps can be 1 GB. Slow pulls = slow deploys. Use multi-stage builds and minimal base images.

Tip

Tag images with the **git SHA**, not `:latest`. Latest is mutable — what it points to changes silently over time. SHA tags are immutable and traceable to a specific commit.

<code>myregistry/myapp:sha-abc1234</code>	<code>immutable, traceable</code>
<code>myregistry/myapp:latest</code>	<code>mutable, no history</code>

Interview line

“We tag images by git short SHA — `sha-abc1234`. Immutable, traceable, makes rollback to a specific commit trivial: change the values.yaml tag, let ArgoCD reconcile.”

Dockerfile

Plain English

A recipe for building a container image. A text file with build instructions.

Technical definition

A text file with instructions (`FROM`, `RUN`, `COPY`, `CMD`, etc.) executed sequentially by `docker build` or compatible tools (kaniko, buildah, BuildKit). Each instruction creates a layer.

Why it matters

Reproducible image builds. The Dockerfile is the source of truth — anyone can rebuild your image from it.

Gotcha

Layer ordering matters for cache efficiency. Put rarely-changing things (base image, system packages) at the top. Frequently-changing things (your code) at the bottom. Maximizes cache hits.

Tip

Multi-stage builds:

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:20-alpine AS runtime
WORKDIR /app
USER node
COPY --from=deps /app/node_modules ./node_modules
COPY . .
CMD ["node", "server.js"]
```

Final image doesn't include build tools. Smaller, fewer CVEs.

Interview line

“I'd review a Dockerfile for: specific base image tag (not `:latest`), multi-stage builds for small final images, non-root user, layer ordering for cache efficiency, no secrets baked in. Red flags: `chmod 777`, `apt install` without `rm -rf /var/lib/apt/lists/`, no `EXPOSE`, no `HEALTHCHECK`.”

Layer

Plain English

One step in a Docker image build. Each `RUN` or `COPY` in your Dockerfile creates a new layer stacked on top.

Technical definition

A filesystem diff produced by a single Dockerfile instruction. Layers are immutable, hash-addressed, and stacked via union filesystems to produce the container's view of the filesystem at runtime.

Why it matters

Layers are cached. If line N didn't change, layers N are reused. Smart Dockerfile design = fast builds.

Gotcha

Each RUN creates a layer. Many RUN lines = many layers = bigger image. Chain commands with `&&`:

```
# BAD - 3 layers
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# GOOD - 1 layer
RUN apt-get update && \
    apt-get install -y curl && \
    rm -rf /var/lib/apt/lists/*
```

Tip

Inspect layers with `docker history <image>`. See which Dockerfile instruction made each layer, and how big it is. Helps identify bloat.

Interview line

“I optimize Dockerfile layer ordering by frequency of change — base image and system packages first, app dependencies next, app code last. Means most of the build benefits from layer cache.”

Multi-stage build

Plain English

Build in one container, run in another. Throw away the build tools in the final image so it stays small.

Technical definition

A Dockerfile with multiple `FROM ... AS <name>` directives. `COPY --from=<stage>` copies files between stages. Only the final stage becomes the published image.

Why it matters

Build tools (compilers, build deps) bloat images and add CVEs. Multi-stage gives you a clean runtime image with just what you need to run.

Gotcha

Forgetting `COPY --chown=user:user` means files end up owned by root, breaking non-root runtime containers.

Tip

Common pattern for Node.js: build in node:alpine, serve in nginx:alpine.

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build          # produces /app/dist

FROM nginx:alpine AS runtime
COPY --from=build /app/dist /usr/share/nginx/html
```

Final image is just nginx + your built static files. No Node.js, no source code, no node_modules.

Interview line

“Multi-stage build is essential for fast deploys and small attack surface. Build stage has the toolchain; runtime stage has only the artifacts. Frontend example: build with Node, serve with nginx. Final image is ~50 MB instead of 1 GB.”

Registry

Plain English

Where Docker images live online. Docker Hub for public, ACR/ECR/GCR for cloud-specific, Harbor for self-hosted.

Technical definition

A server speaking the OCI distribution spec, hosting images at namespaced paths. Supports push/pull operations, authentication, vulnerability scanning, image signing.

Why it matters

Pull from registry on every deploy. Registry speed and reliability matter. Cloud-local registries (ACR for AKS) pull faster than internet registries (Docker Hub).

Gotcha

Docker Hub has rate limits for anonymous pulls (100 per 6h). Anonymous pulls from CI can hit this fast. Authenticate or use a mirror.

Tip

For Kubernetes, use a registry close to the cluster:

- AKS + ACR
- EKS + ECR

- GKE + Artifact Registry

Same region, free pulls (no egress cost).

Interview line

“For AKS we use ACR in the same region. Same-region image pulls are free, faster, and the AKS kubelet identity gets AcrPull role via `az aks update --attach-acr` — no image pull secrets needed.”

Image digest

Plain English

A cryptographic fingerprint of an image. Truly unique. Tags can change; digests can't.

Technical definition

A SHA-256 hash of the image's manifest. Written as `myimage@sha256:abc....`. Two pulls of the same digest are guaranteed identical, byte for byte.

Why it matters

Tags are mutable. Someone can push `myapp:1.2.3` again with different content. Digests can't change. For production-grade immutability, pin to digest.

Gotcha

Digest pinning is ugly to read:

`image: myacr.azurecr.io/myapp@sha256:b3a8c7e5d4f2a1b0c9e8d7f6a5b4c3d2e1f0a9b8c7d6e5f4a3b2c1d0e`

Hard for humans, but exactly what production needs.

Tip

Use a tool (Renovate, Dependabot) to auto-PR digest updates. Humans manage tag semantics; bots manage digest pinning.

Interview line

“We're moving from tag-pinning to digest-pinning so a malicious registry push can't poison our deploys. We'd use Renovate to auto-PR digest bumps. Tag for human readability, digest for cryptographic guarantee.”

Section F — Kubernetes

Pod

Plain English

One or more containers that live together on the same node, sharing network and storage. The smallest deployable unit.

Technical definition

The smallest deployable unit in Kubernetes. A Pod is a scheduling boundary — all containers in a Pod run on the same node, share the network namespace (same IP, same port space), and can share volumes.

Why it matters

Single-container Pods are the common case. Multi-container Pods use the “sidecar pattern” — logging agents, mesh proxies (Istio’s istio-proxy), init logic.

Gotcha

You almost never create Pods directly in production. Use Deployments, StatefulSets, or Jobs. These higher-level objects handle restarts, scaling, and updates.

Tip

Use init containers for one-time setup before the main containers start:

```
spec:
  initContainers:
    - name: wait-for-db
      image: busybox
      command: ['sh', '-c', 'until nc -z db 5432; do sleep 1; done']
  containers:
    - name: app
      image: myapp
```


Interview line

“Pods share network and storage — that’s what enables the sidecar pattern. Istio’s mesh works because the istio-proxy sidecar intercepts ALL traffic for the main container, transparently. They communicate via localhost.”

Node

Plain English

A worker machine in the cluster. Could be a VM or physical. Runs your Pods.

Technical definition

A VM (or physical) machine joined to the Kubernetes cluster. Runs:

- **kubelet** — talks to the API server, manages pod lifecycle
- **kube-proxy** — implements Service networking
- **Container runtime** (containerd, CRI-O)

Why it matters

Pods run on nodes. Cluster capacity is the sum of node capacity (minus system overhead).

Gotcha

More smaller nodes — less complexity. Fewer larger nodes are easier to manage (fewer API objects, less control-plane load) but worse for bin-packing (one big idle pod wastes more capacity).

Tip

Plan for ~80% utilization at peak. Above that, scheduler struggles with fragmentation. Below that, you’re wasting money.

Interview line

“Node sizing is a tradeoff. Fewer larger nodes simplify management but worsen bin-packing. More smaller nodes improve bin-packing but increase control-plane load. We picked 2× D2s_v3 — fits our Free Trial quota and gives ~60-70% utilization headroom.”

Namespace

Plain English

A folder inside the cluster. Lets you separate things logically — **dev** namespace, **prod** namespace, **monitoring** namespace.

Technical definition

A scope for names. Most objects (Pods, Services, ConfigMaps) are namespaced; some (Nodes, ClusterRoles, PersistentVolumes) are cluster-scoped. Namespaces don't isolate networking by default — pods in **dev** can talk to pods in **prod** unless you add NetworkPolicies.

Why it matters

Logical separation between teams, environments, or apps. RBAC, ResourceQuotas, and LimitRanges attach to namespaces. Easy logical boundary for “this is dev, that is prod.”

Gotcha

Namespaces are NOT a security boundary by default. Use NetworkPolicies to prevent cross-namespace traffic. Use RBAC to prevent cross-namespace access.

Tip

Use namespaces aggressively. They're free. Even in a small cluster, namespaces help organize.

Interview line

“Namespaces are logical, not security. For multi-tenancy I'd combine namespaces with NetworkPolicies, ResourceQuotas, and OPA/Kyverno policies. Just namespacing isn't isolation.”

Deployment

Plain English

A way to run N copies of a Pod and update them safely. The default for stateless apps.

Technical definition

A higher-level object that manages a ReplicaSet, which manages Pods. Provides:

- Rolling updates with configurable max-surge / max-unavailable
- Rollback to previous revisions
- Selector-based pod ownership

Why it matters

The standard way to run web servers, API backends, workers — anything stateless.

Gotcha

replicas: 3 doesn't always mean 3 are serving traffic. Readiness probes determine “ready to receive requests.” Pods can be running but not ready.

Tip

Use a poison-pill rolling-update strategy: `maxUnavailable: 0` ensures you never have fewer pods than desired during a deploy. `maxSurge: 1` allows one extra pod during deploy.

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

Interview line

“For stateless apps, Deployment with rolling update. For risky deploys, I’d switch to Argo Rollouts blue/green or canary. For stateful workloads, StatefulSet. The choice depends on tolerance for partial-traffic states and identity needs.”

ReplicaSet

Plain English

“Make sure exactly N copies of this Pod exist.” Usually managed by a Deployment, not created directly.

Technical definition

A controller that ensures the desired number of Pod replicas matching a selector. Each Deployment owns multiple ReplicaSets — one per revision. Old ReplicaSets are kept around (scaled to 0) for rollback.

Why it matters

Behind every Deployment is a ReplicaSet. Looking at ReplicaSets helps debug Deployment behavior.

Gotcha

`kubectl rollout history` shows revisions. Each is a separate ReplicaSet. Default keep-count is 10. Configure `revisionHistoryLimit` to keep more (or fewer) for rollback options.

Tip

When debugging weird Deployment behavior, look at ReplicaSets:

```
kubectl get rs -n dev
```

If you see two ReplicaSets with non-zero replicas, you’re in the middle of a rollout (or a stuck one).

Interview line

“ReplicaSets are managed by Deployments, but I check them when debugging stuck rollouts. Two non-zero ReplicaSets means rollout is in progress. The pause might be readiness probe failures on the new ReplicaSet.”

StatefulSet

Plain English

Like Deployment but for things with identity — databases, brokers, leader-elected systems. Pods get stable names like `db-0`, `db-1`.

Technical definition

Manages Pods with:

- **Stable network identity** — `<statefulset>-<ordinal>.<service>`
- **Stable storage** — each ordinal gets its own PVC via `volumeClaimTemplates`
- **Ordered start** — 0 starts, becomes Ready, then 1 starts
- **Reverse delete** — 2 deleted, then 1, then 0

Why it matters

Databases and stateful systems need predictable identity. A Postgres primary at `db-0` is reliably the same Pod across restarts; its data persists in `data-db-0` PVC.

Gotcha

Scaling down a StatefulSet doesn’t delete the PVCs. Data sticks around until you delete the PVCs explicitly. This is by design — accidental scale-down doesn’t lose data.

Tip

For databases, prefer a managed service (Azure Database for PostgreSQL, RDS) over self-hosted StatefulSets. The operational burden of self-hosted DBs is significant.

Interview line

“Postgres on a StatefulSet works for a lab. For production I’d use Azure Database for PostgreSQL — managed backups, point-in-time restore, automated patching, multi-AZ. The operational burden of self-hosting databases on Kubernetes is high.”

Service

Plain English

A stable network endpoint for a set of Pods. Pods come and go; the Service stays.

Technical definition

A Kubernetes object with a selector and ports. kube-proxy on each node programs iptables/IPVS rules to load-balance Service traffic across matching Pods.

Types:

- **ClusterIP** — internal IP only (default)
- **NodePort** — open same port on every node's external IP
- **LoadBalancer** — cloud LB pointing at NodePort
- **ExternalName** — DNS CNAME to an external service

Why it matters

Stable IP for variable Pods. Pods can be deleted/recreated; the Service IP doesn't change.

Gotcha

Service load balancing is iptables/IPVS — pseudo-random distribution. No request affinity by default. For sticky sessions, use `sessionAffinity: ClientIP` (basic) or a service mesh (better).

Tip

Default ClusterIP is enough for most cases. Use Ingress or Gateway for external traffic, not LoadBalancer Services (saves cloud LB cost).

Interview line

“Most Services should be ClusterIP. External access goes through one Ingress or Istio gateway pointing at all internal Services. One LoadBalancer per Service gets expensive and fragmented.”

ConfigMap

Plain English

A bag of key-value config for your app. Decouples config from the container image.

Technical definition

A namespaced object holding string data. Mountable as files or env vars in Pods.

Why it matters

Same image, different configs per environment. The image is portable; the ConfigMap is environment-specific.

Gotcha

ConfigMap data isn't encrypted at rest unless you've enabled etcd encryption. Don't use ConfigMaps for secrets — use Secret objects (which are also weakly protected; for real secrets, use External Secrets + Key Vault).

Tip

For large configs, mount as files (one file per key). For a few keys, env vars are fine.

```
spec:
  containers:
    - name: app
      envFrom:
        - configMapRef:
            name: app-config
      volumeMounts:
        - name: config-files
          mountPath: /etc/app/conf.d
  volumes:
    - name: config-files
      configMap:
        name: app-config
```

Interview line

“Config in ConfigMaps, secrets in External Secrets pulling from Key Vault. ConfigMap data isn't encrypted by default and gets logged on apply errors. Secrets are base64 (also not encryption) but at least are tagged for special handling.”

Secret

Plain English

A bag of sensitive config (passwords, API keys, certificates) for your app.

Technical definition

Like ConfigMap, but base64-encoded and tagged for special handling. Optionally encrypted at rest in etcd (requires explicit configuration).

Why it matters

Keeps sensitive data out of container images. Mountable as env vars or files.

Gotcha

“Base64-encoded” is NOT encryption. Anyone with read access to the Secret can decode it. Real production uses External Secrets Operator pulling from Key Vault, AWS Secrets Manager, or

HashiCorp Vault.

Tip

Enable etcd encryption-at-rest on your cluster:

```
az aks update --resource-group rg --name cluster --enable-secret-rotation
```

Easy to forget; significant security win.

Interview line

“Kubernetes Secrets are base64 — that’s not encryption. For real secrets in production I’d use External Secrets Operator pulling from Azure Key Vault. The pod sees a Secret; the actual secret value lives in Key Vault with audit logging, rotation, and IAM.”

PersistentVolume / PersistentVolumeClaim

Plain English

Persistent storage that survives Pod restarts. PV is the actual storage; PVC is a request for storage.

Technical definition

- **PV** (PersistentVolume) — actual storage resource (Azure disk, NFS export, etc.), provisioned by a StorageClass dynamically or statically defined.
- **PVC** (PersistentVolumeClaim) — a request for storage with size and access mode. Gets bound to a matching PV.

Access modes:

- **ReadWriteOnce** (RWO) — one node at a time. Standard for block storage (Azure disk, EBS)
- **ReadOnlyMany** (ROX) — many nodes can read
- **ReadWriteMany** (RWX) — many nodes can write. Needs NFS or Azure Files

Why it matters

Without PV/PVC, restarting a Pod loses its filesystem state. Databases, file uploads, anything stateful needs persistent storage.

Gotcha

ReadWriteOnce means one node at a time. Two Pods on different nodes can’t share an RWO volume. Use RWX (Azure Files, NFS) if needed, with performance and cost tradeoffs.

Tip

In our lab:

- Postgres: managed-csi (Azure Disk, RWO) — one Pod, fast
- npm/trivy cache: azurefile-csi (Azure Files, RWX) — shared across CI Pods

Interview line

“Storage class choice depends on access pattern. RWO Azure Disk for databases (fast, one node). RWX Azure Files for shared caches (slower, multi-node). For real production data I’d use a managed database, not a StatefulSet on a PVC.”

Ingress / Gateway

Plain English

The “front door” of the cluster. Routes public URLs to internal Services.

Technical definition

- **Ingress** — older API, simple HTTP routing rules
- **Gateway API** — newer, more expressive (supports TCP/UDP, splits responsibilities)
- **Istio Gateway + VirtualService** — service mesh equivalent

All define how external traffic reaches internal Services.

Why it matters

Without it, every Service needs its own LoadBalancer (expensive) or NodePort (ugly). With it, one LB → ingress controller → many Services based on Host header or path.

Gotcha

Different ingress controllers (nginx-ingress, traefik, Istio gateway, Azure Application Gateway Ingress Controller) have different annotations and feature sets. Picking the right one matters.

Tip

Use one ingress controller per cluster (cheaper, simpler) and route to all Services through it. Istio Gateway is good for service-mesh-aware deployments.

Interview line

“For service mesh deployments, Istio Gateway. For non-mesh, nginx-ingress or Application Gateway Ingress Controller (which gives you a real Azure App Gateway). One ingress for the whole cluster, routes to Services by Host header.”

ServiceAccount

Plain English

A workload’s identity. Answers “who is this Pod when it talks to the Kubernetes API?”

Technical definition

A namespaced Kubernetes object representing a non-human identity. Bound to RBAC roles. Mounted into Pods as a JWT token at `/var/run/secrets/kubernetes.io/serviceaccount/token`.

Why it matters

Every Pod has a ServiceAccount (default `default` if not specified). Used for:

- Talking to the Kubernetes API
- Workload Identity (Azure) — mapping to an Azure AD app
- IRSA (AWS) — mapping to an IAM role

Gotcha

Never use the `default` ServiceAccount. It has the default permissions, which may be more than your Pod needs. Always create a specific SA per workload.

Tip

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: myapp
  namespace: prod
  annotations:
    azure.workload.identity/client-id: <azure-ad-app-client-id> # Workload Identity
```

Interview line

“Every workload gets its own ServiceAccount, bound to a specific Role with least privilege. For cloud access — like ACR pulls or Key Vault reads — I use Workload Identity to federate the SA to an Azure AD app. No secrets to manage.”

RBAC (Role-Based Access Control)

Plain English

Who can do what in the cluster. “Alice can read pods in `dev`, but not edit them. The Jenkins SA can do helm upgrade in `dev` and `prod`.”

Technical definition

Roles (sets of permissions on resources) bound to subjects (users, groups, ServiceAccounts) via RoleBindings (namespace-scoped) or ClusterRoleBindings (cluster-wide).

Why it matters

Least privilege. Limits damage from compromised credentials. Auditing requires this.

Gotcha

ClusterRole + ClusterRoleBinding is cluster-wide. Easy to over-grant. Prefer Role + RoleBinding (namespace-scoped) whenever possible.

Tip

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

Start with no permissions. Add only what's actually needed. Audit quarterly.

Interview line

“Least privilege per ServiceAccount. ClusterRoles only when truly cluster-wide (CRD controllers, ingress controllers). Audit logging via Kubernetes API audit logs to Log Analytics. Quarterly access review with the security team.”

Controller

Plain English

A program that watches the cluster and reconciles state — if reality doesn't match desired, the controller makes it match.

Technical definition

A control loop running against the Kubernetes API. Watches objects, computes diffs between actual and desired state, takes actions to converge. Built-in controllers (DeploymentController, ReplicaSetController) ship with Kubernetes; custom controllers (Argo Rollouts, ArgoCD) are deployed as workloads.

Why it matters

The “Kubernetes way” — declarative, eventually consistent. You declare what you want; controllers make it so.

Gotcha

Controllers fight each other if poorly designed. If two controllers manage the same resource, they can ping-pong. Use ownership references and selector scoping to avoid this.

Tip

When something doesn't match what you applied:

1. Check if a controller is reverting your change (`kubectl describe` shows recent events)
2. Check ownership — `kubectl get <resource> -o yaml | grep ownerReferences`
3. Check the controller's logs — most controllers log decisions

Interview line

“Argo Rollouts is a custom controller with its own CRD. Watches Rollout objects, manages ReplicaSets and Services directly. Same pattern as the built-in DeploymentController — control loop reconciling actual to desired state.”

CRD (Custom Resource Definition)

Plain English

A way to define your own Kubernetes object types. Adds new “kinds” to the cluster API.

Technical definition

A YAML manifest registering a new API kind with schema, validation, and printer columns. Coupled with one or more controllers that know what to do with instances of that kind.

Why it matters

The whole CNCF ecosystem builds on CRDs:

- Argo Rollouts → `Rollout`, `AnalysisTemplate`
- ArgoCD → `Application`, `ApplicationSet`
- Istio → `VirtualService`, `DestinationRule`, `Gateway`
- Prometheus Operator → `Prometheus`, `ServiceMonitor`, `PrometheusRule`

Gotcha

CRDs are cluster-scoped (the definition). You can't have v1 of a CRD in one namespace and v2 in another. Upgrading CRDs across versions sometimes requires migration.

Tip

Look at the CRDs installed in your cluster:

```
kubectl get crd
kubectl explain rollout      # see the schema
```

Interview line

“CRDs are how you extend Kubernetes. Argo Rollouts, ArgoCD, Istio, cert-manager — all of them ship CRDs and a controller. The pattern is: declarative resource + controller that reconciles. Same as built-in objects.”

Operator

Plain English

A custom controller that knows about a specific application — installs it, upgrades it, backs it up, scales it.

Technical definition

CRD(s) + a controller that encodes operational expertise for a specific application. Goes beyond simple reconciliation — handles versioned upgrades, day-2 ops like backup/restore, application-specific autoscaling.

Examples

- Prometheus Operator — manages Prometheus, Alertmanager, ServiceMonitor
- cert-manager — issues and rotates TLS certs
- ArgoCD itself
- Postgres Operator (Zalando, Crunchy)

Why it matters

If your team manages something complex (databases, brokers, ML platforms) on Kubernetes, an Operator can codify “how to operate this” so anyone can install/upgrade safely.

Gotcha

Writing a good Operator is hard. Many teams over-engineer Operators when a Helm chart would do. Operators are worth it when you have non-trivial Day-2 ops (versioned upgrades, backups, autoscaling).

Tip

If you find yourself writing a Helm chart with complex **pre-upgrade-hook** Jobs that handle “if upgrading from version X, run job Y” — that’s a sign you want an Operator.

Interview line

“Helm for installs of stateless software. Operators for complex stateful systems with version-specific upgrades. The OperatorHub catalog has 200+ pre-built. We’d write our own only for very specific in-house needs.”

Section G — Terraform

Infrastructure as Code (IaC)

Plain English

Describe infrastructure in text files instead of clicking around in cloud consoles. Run a command, the infrastructure gets created.

Technical definition

A practice of provisioning and managing infrastructure through declarative or imperative code rather than manual processes. Tools include Terraform (multi-cloud), Pulumi (real programming languages), CloudFormation (AWS), ARM/Bicep (Azure native), CDK (real languages on top of CloudFormation/Terraform).

Why it matters

- **Reproducible** — same code produces same infra
- **Reviewable** — diffs in PRs
- **Versioned** — git history shows who changed what
- **Disaster recovery** — destroy and rebuild from scratch
- **Self-documenting** — code IS the documentation

Gotcha

Don't mix click-ops and IaC. If you `terraform apply` to create something, then change it in the console, IaC drift is invisible until next apply (which may revert your changes).

Tip

Treat IaC like application code — PRs, code review, CI for `terraform plan`, branch protection. The same discipline.

Interview line

“We use Terraform because it's cloud-agnostic, has the largest provider ecosystem, and a clear declarative model. For Azure-only projects, Bicep is the more native choice. CDK is for teams that hate HCL and want real programming language constructs.”

State (Terraform state)

Plain English

A file mapping your code to real cloud resources. Without it, Terraform doesn't know what already exists.

Technical definition

A JSON file (`terraform.tfstate`) tracking the mapping between Terraform resource addresses and real-world resource IDs, plus attributes. Required for diff computation.

Why it matters

State IS Terraform's source of truth for "what exists." Without it, Terraform thinks nothing exists and tries to create everything.

Gotcha

State contains secrets — passwords, keys, sensitive outputs. Encrypt at rest. Never commit to git.

Tip

Add `*.tfstate` and `*.tfstate.backup` to `.gitignore`. Always.

For team work, use a remote backend (Azure Storage, S3) so everyone shares one state file.

Interview line

"State is the source of truth for Terraform. Remote backend with locking (Azure blob lease, S3+DynamoDB) for team work. Encrypt at rest. Backup before risky operations. State contains secrets, so access control matters."

Remote backend

Plain English

Store Terraform state in the cloud, not on your laptop. Lets your team share state and prevents simultaneous applies.

Technical definition

A state storage location supporting concurrent access with locking. Common backends:

- **azurerm** — Azure Storage Account
- **s3** — AWS S3 + DynamoDB for locking
- **gcs** — Google Cloud Storage
- **remote** — Terraform Cloud / Enterprise

Why it matters

Without remote backend, only one person can work on the infrastructure. State on laptop = lost if laptop dies.

Gotcha

Configuring the backend requires the backend to exist. Chicken-and-egg: you need a Storage Account to use a Storage Account backend. Solve with a “bootstrap” step that uses local backend to create the SA, then migrate live infra to use it.

Tip

```
terraform {  
  backend "azurerm" {  
    resource_group_name = "tfstate-rg"  
    storage_account_name = "mytfstate12345"  
    container_name       = "tfstate"  
    key                  = "prod/terraform.tfstate"  
  }  
}
```

Use the `key` to namespace state by environment or service.

Interview line

“Two Terraform configs in the lab: `bootstrap/` with local backend creates the Storage Account; `live/` uses that SA as remote backend. Chicken-and-egg solution. Bootstrap rarely changes after first run.”

Provider

Plain English

A plugin Terraform uses to talk to a specific cloud or service. Each cloud has its own provider.

Technical definition

A Go binary that translates Terraform HCL resource declarations into API calls. `hashicorp/azurerm`, `hashicorp/aws`, `hashicorp/kubernetes`, `hashicorp/helm`, hundreds more.

Why it matters

The provider is what makes Terraform multi-cloud. Same HCL syntax, different providers for different clouds.

Gotcha

Provider versions matter. New provider versions can have breaking changes. Always pin:

```
required_providers {
  azurerm = {
    source  = "hashicorp/azurerm"
    version = "~> 3.0"
  }
}
```

~> 3.0 means “any 3.x, never 4.x” — safe for minor updates.

Tip

Run `terraform init -upgrade` periodically to pull provider updates within your constraints. Review the changelog.

Interview line

“Providers are versioned plugins. Always pin with `version = '~> 3.0'` to avoid surprise breaks. Major version bumps usually have migration notes. We update providers monthly via Renovate-style automation.”

Module

Plain English

A reusable bundle of Terraform code. Define an “aks-cluster” module once, use it in dev, staging, prod.

Technical definition

A directory of `.tf` files invocable from other configurations via `module "name" { source = "..."`. Modules accept inputs (variables) and produce outputs.

Why it matters

DRY. Don’t repeat AKS configuration in 3 environment directories — make it a module, call it 3 times with different inputs.

Gotcha

Modules can be too generic (lots of variables, hard to use) or too specific (hard-coded values, hard to reuse). Find the right level.

Tip

Organize modules by responsibility:

```
modules/
  network/      # VNet, subnets, NSGs
  acr/          # Container registry
```



```
aks/           # Kubernetes cluster
monitoring/    # Log Analytics
```

Versioned modules in a private Terraform Registry are next-level. Refer to them by version tag:
`source = "git::ssh://.../modules/aks?ref=v1.2.3".`

Interview line

“We have three modules: network, acr, aks. Each takes inputs from the calling config. For team usage at scale, I’d publish them to a private registry with semver tags, so consumers pin versions and get a predictable upgrade path.”

State locking

Plain English

“Don’t let two people change state at the same time.” Prevents corruption.

Technical definition

The backend acquires a lock before `terraform apply` or `terraform plan -lock=true`. Releases on completion. Other users see “state is locked” until the lock is released.

Why it matters

Without locking, two simultaneous applies can corrupt state — both write their changes, but only one wins, and the state file no longer reflects reality.

Gotcha

Crashed applies can leave stale locks. `terraform force-unlock <id>` removes them. Use cautiously — make sure no one else is running.

Tip

Azure Storage blob lease handles this automatically. S3 requires a DynamoDB table for the lock state.

Interview line

“State locking via the backend — Azure blob lease, AWS DynamoDB. Critical for team work. We’ve never had to force-unlock, but the procedure is well-documented in the runbook.”

Data sources

Plain English

Read existing cloud resources without managing them. Like `SELECT`, not `CREATE`.

Technical definition

A **data** block that queries existing resources at plan time and exposes their attributes for use elsewhere in the configuration.

Why it matters

Reference resources you don't own. Reference resources created by other Terraform configs. Get the current subscription ID, tenant ID, etc.

Tip

```
data "azurerm_subscription" "current" {}

resource "azurerm_role_assignment" "example" {
  scope      = data.azurerm_subscription.current.id
  role       = "Reader"
  principal_id = "..."}
}
```

Interview line

“Data sources for resources I don't own — looking up an existing Key Vault by name, getting the subscription ID, finding a VNet created by the platform team. Keeps responsibility boundaries clean.”

Plan vs Apply

Plain English

plan shows what would change. **apply** does it. Always plan first.

Technical definition

- **terraform plan** — refreshes state, compares config to state, computes diff, shows what would change
- **terraform apply** — runs plan, prompts for confirmation, executes the diff
- **terraform apply tfplan** — applies a saved plan, no prompt

Why it matters

Plan-first is mandatory in production. Surprises in CI = outages. Save the plan and apply it for deterministic deploys.

Tip

```
terraform plan -out=tfplan
# review
terraform apply tfplan
```

Save the plan to a file and pass it to apply. Guarantees no surprise changes between plan and apply.

Interview line

“Always plan first. Save with `-out=tfplan`, then `apply tfplan` — guarantees no surprise changes between plan and apply. In CI, we plan on PR, apply on merge to master. Manual approval gates between.”

Drift

Plain English

Difference between what Terraform thinks exists and what really exists. Caused by people clicking in the console after Terraform set things up.

Technical definition

When real cloud resources are modified outside Terraform (manual edits, other tools, automatic cloud actions), state goes out of sync with reality.

Why it matters

Drift = surprises on next apply. Terraform may revert manual fixes, or fail because resources are in unexpected states.

Gotcha

Some drift is unavoidable — cloud services auto-update some attributes (e.g., AKS node labels, system tags). Terraform plans show these every time. Use `ignore_changes` to suppress noise.

Tip

Run drift detection in CI nightly:

```
terraform plan -detailed-exitcode  
# exit 0 = no changes, 2 = changes, 1 = error
```

If exit 2, alert. Drift happened somewhere.

Interview line

“Drift detection nightly in CI. If `plan -detailed-exitcode` returns 2, alert. Could be legit manual fix that needs codifying, or unauthorized change that needs reverting. Both deserve human review.”

Section H — Helm

Helm

Plain English

A package manager for Kubernetes. Like apt or yum, but for Kubernetes apps.

Technical definition

Templates + values combine to produce Kubernetes manifests. A package (chart) is installed as a versioned release tracked in cluster state.

Why it matters

Don't reinvent install/upgrade logic for every app. Use community charts for common software (Prometheus, Nginx Ingress, cert-manager).

Gotcha

Helm templates are Go templates with quirks. Indentation matters. Conditional logic is verbose. Errors can be cryptic.

Tip

Use Helm for installs of third-party software. For in-house apps, the choice between Helm and Kustomize is taste.

Interview line

“Helm for community charts (Prometheus, Istio, cert-manager). Kustomize for in-house apps. Most teams end up with both. Helm's complexity is the price of reusability.”

Chart

Plain English

A package of Kubernetes manifests with parameters.

Technical definition

Directory with `Chart.yaml` (metadata), `values.yaml` (default values), `templates/` containing Go-templated YAML, and optionally `charts/` (sub-charts).

Why it matters

The unit of distribution and versioning in the Helm world.

Tip

Linting catches most chart issues:

```
helm lint ./mychart
helm template ./mychart    # render to see actual YAML
```

Interview line

“Charts are versioned packages. Public ones on Artifact Hub, private ones in OCI registries (ACR supports this). Same versioning rigor as application code — semver, changelog, breaking changes documented.”

Release

Plain English

An installation of a chart with a specific name. You can install the same chart multiple times under different names.

Technical definition

A named, versioned deployment of a chart in a namespace. Each `helm install` creates a Release tracked in cluster state (as Secrets by default).

Why it matters

Release tracking enables `helm upgrade` and `helm rollback`. Helm knows what’s installed and what was changed.

Tip

Use `helm upgrade --install` instead of `helm install` — idempotent. Installs if missing, upgrades if exists. Common pattern in CI.

Interview line

“In CI, always `helm upgrade --install` — idempotent. First run installs, subsequent runs upgrade. Eliminates ‘release already exists’ errors on retries.”

Values

Plain English

Variables you pass to a chart. Different values per environment.

Technical definition

YAML data merged with chart defaults (`values.yaml`) to produce the final rendered manifests.

Why it matters

Same chart, different deployments. `values-dev.yaml` and `values-prod.yaml` override defaults for environment specifics.

Tip

Order of precedence (highest wins):

1. `--set key=value` on command line
2. `--values custom.yaml`
3. chart's `values.yaml`

CI typically uses `--values` for environment files plus `--set image.tag=<sha>` for the build-specific image tag.

Interview line

“Helm value precedence: command-line `--set` > `--values file` > chart defaults. In CI we use `--values values-prod.yaml --set image.tag=sha-abc1234`. Environment defaults from the file, build-specific bits via `--set`.”

Umbrella chart

Plain English

A chart that depends on other charts. Lets you compose complex apps from smaller charts.

Technical definition

A parent chart with `dependencies:` in `Chart.yaml`. Sub-charts under `charts/`. Helm renders the parent + all dependencies in one install.

Why it matters

Compose complex apps (frontend + backend + database) into one Helm release. Single install/upgrade, single rollback.

Gotcha

Sub-chart values are scoped under the sub-chart name in the parent values:

```
# Parent values.yaml
frontend:           # this passes to the frontend sub-chart
  replicas: 3
backend:            # this passes to the backend sub-chart
  replicas: 5
```

Tip

Pin sub-chart versions in the parent's Chart.yaml. Avoids surprise breaks when the sub-chart releases new versions.

Interview line

“Our three-tier chart is umbrella with frontend, backend, database sub-charts. One **helm upgrade --install** deploys everything together. Each sub-chart can be tested in isolation but the umbrella is what production uses.”

Section I — Azure

Service Principal (SP)

Plain English

A non-human identity for automation tools — Terraform, CI/CD pipelines, scripts. Has a client ID and a secret.

Technical definition

An Azure AD application object with a credential (client secret or certificate). RBAC role assignments grant it permissions in subscriptions/resource groups.

Why it matters

External tools (CI runners, on-prem scripts) need an identity. SP is the explicit answer.

Gotcha

SP credentials are long-lived secrets. If leaked, the attacker has whatever permissions the SP has. Rotate them. Or use Workload Identity instead (no secrets).

Tip

```
az ad sp create-for-rbac \  
  --name sp-myapp-terraform \  
  --role Contributor \  
  --scopes /subscriptions/<sub-id>
```

Save the output JSON — it has the only copy of the client secret.

Interview line

“SP for tools outside Azure — our Terraform CI. Inside Azure (AKS pods), I use Workload Identity to federate the pod’s ServiceAccount to an Azure AD app. No secrets to manage, no rotation.”

Managed Identity

Plain English

An automatically managed identity for Azure resources. The cloud handles credentials.

Technical definition

Azure resources (VMs, App Service, AKS) get an identity managed by Azure. Two types:

- **System-Assigned** — tied to the resource lifecycle
- **User-Assigned** — independent identity, can be assigned to multiple resources

Why it matters

No credentials to store, no rotation to manage. Best practice for Azure-internal authentication.

Tip

Always prefer Managed Identity over Service Principal when possible. SPs are a fallback for things outside Azure.

Interview line

“Inside Azure: Managed Identity. The VM/AKS/Function gets an identity automatically. The cloud rotates credentials. Outside Azure: Service Principal as the fallback. Best practice is to minimize SP usage.”

Workload Identity (AKS)

Plain English

Pods get Azure AD identities without storing secrets. AKS-specific feature.

Technical definition

OIDC federation — Azure trusts JWT tokens issued by your AKS cluster’s OIDC issuer. Pods get a federated token, exchange it for an Azure AD access token via Azure AD’s token endpoint.

Why it matters

No long-lived credentials in the pod. The cloud handles trust via OIDC.

Gotcha

Requires AKS to have OIDC issuer and workload identity enabled. Older clusters need an upgrade. Each Azure AD app needs a federated credential pointing at the SA.

Tip

The setup is three things:

1. AKS has `oidc-issuer` enabled
2. Azure AD App has a federated credential for `system:serviceaccount:<ns>:<sa-name>`
3. ServiceAccount has `azure.workload.identity/client-id: <app-id>` annotation

Interview line

“Workload Identity is the modern way. OIDC federation, no secrets, automatic. Production target. Our lab still uses SP credentials in some places — a known gap I’d address before production.”

AKS (Azure Kubernetes Service)

Plain English

Managed Kubernetes on Azure. Azure runs the control plane, you manage worker nodes.

Technical definition

Azure’s managed Kubernetes offering. Control plane is free in the standard SKU (no SLA) or paid in the Premium SKU (99.95% uptime SLA). Worker nodes are billed as VMs.

Why it matters

Skip the operational burden of running Kubernetes control plane (etcd backups, certificate rotation, API server upgrades). Focus on workloads.

Tip

`az aks stop` saves money — control plane stays free, nodes don’t bill. Great for dev/lab clusters.

Interview line

“AKS for managed Kubernetes on Azure. Free control plane in standard SKU. We use `az aks stop` for our lab cluster overnight — saves ~75% on costs without losing setup.”

ACR (Azure Container Registry)

Plain English

Docker image hosting on Azure. Like Docker Hub but private, faster (in your region), and more secure.

Technical definition

OCI-compliant container registry. Three SKUs: Basic (\$5/mo), Standard (\$20/mo, faster), Premium (\$300/mo, geo-replication + advanced features).

Why it matters

Same-region pulls are free and fast for AKS. AcrPull role assignment to AKS kubelet identity = no image pull secrets needed.

Tip

```
az aks update --attach-acr <acr-name>
```

This grants the AKS kubelet identity AcrPull role on the ACR. After this, the cluster can pull images without explicit credentials.

Interview line

“ACR Basic SKU is enough for most needs. `az aks update --attach-acr` wires permissions so the cluster pulls without image pull secrets. For multi-region production, Premium for geo-replication.”

Log Analytics Workspace (LAW)

Plain English

Centralized log storage on Azure. Where logs from all your stuff end up.

Technical definition

Azure Monitor’s underlying log storage. Stores logs from VMs, AKS containers, Azure resources, custom apps. Queried with Kusto Query Language (KQL).

Why it matters

AKS sends container logs via OMS agent. App logs, audit logs, system events all land here. Single pane of glass.

Gotcha

Costs scale with ingestion volume. A noisy app can rack up \$500/month easily. Set retention carefully and use sampling for non-critical logs.

Tip

Set workspace retention (default 30 days, configurable). For audit logs that need long retention, route to a separate workspace with longer retention.

Interview line

“LAW for AKS container logs via OMS agent, app logs, and Azure resource logs. Costs scale with ingestion, so I set retention by log type — security audit logs 90 days, app logs 30. KQL for queries is powerful once you learn it.”

Section J — CI/CD

CI (Continuous Integration)

Plain English

Every code change is automatically built and tested. Catches bugs early.

Technical definition

A pipeline that runs on every commit or pull request: compile, lint, test, package. Reports pass/fail. Often gates merging.

Why it matters

Without CI, bugs accumulate. With CI, every commit either keeps the build green or breaks it (with immediate signal to fix).

Tip

Keep CI fast (<10 min for the critical path). Slow CI = developers wait, batch changes, skip pushes. Test parallelization, build caching, focused test suites.

Interview line

“CI on every PR + every push to master. Critical path under 10 minutes. Caching everywhere — npm deps, Docker layers, test runner caches. Parallel jobs split the matrix. Slow CI is a productivity killer.”

CD (Continuous Delivery vs Deployment)

Plain English

- **Continuous Delivery** — always ready to deploy. A human pushes the deploy button.
- **Continuous Deployment** — auto-deploys on every successful CI. No human gate.

Why it matters

The difference is whether prod deploys are automated. Most teams say “CD” but mean Delivery.

Tip

For a team starting out: Continuous Delivery with manual gates. Once trust is high, automated rollbacks work, and SLO budget exists: move to Continuous Deployment.

Interview line

“We have Continuous Delivery — every commit produces a deployable artifact, but human approval gates the prod deploy. Continuous Deployment would require automated rollback on SLO breach and high test confidence. Achievable, not yet achieved.”

Pipeline

Plain English

An automated sequence of build/test/deploy steps. Defined as code.

Technical definition

Code (Jenkinsfile, `ci-cd.yaml`, `.circleci/config.yml`) defining stages, jobs, and conditions. Stored in the repo alongside the application code.

Why it matters

Pipelines as code = versioned, reviewable, reproducible. UI-only pipelines have no audit trail and no rollback.

Tip

Always have pipelines in the repo. The CI tool reads them from the repo. Avoid pipelines configured in UIs — they don’t survive tool migrations.

Interview line

“Pipelines as code, in the repo. Jenkinsfile for Jenkins, `ci-cd.yaml` for GitHub Actions, `config.yml` for CircleCI. Versioned alongside the app. PR review covers pipeline changes. UI-configured pipelines are an anti-pattern.”

Artifact

Plain English

The output of a build that you keep — image, JAR, ZIP. Tagged and versioned.

Technical definition

A built, immutable output of CI, stored in an artifact registry for traceability and reuse. Container images, archive files, npm packages.

Why it matters

Promote the same artifact through environments. Don't rebuild for prod — that defeats the testing in dev/staging.

Tip

Tag artifacts with git SHA, not build number. SHA is deterministic; build number resets when you change CI tools.

Interview line

“Build once, deploy many. Same image SHA goes dev → staging → prod. Rebuilding per environment means prod runs different bytes than what was tested. That's a real bug source in many orgs.”

GitOps

Plain English

Git is the source of truth for what should be deployed. A controller watches Git and applies changes to the cluster.

Technical definition

An operating model where:

1. Desired state is defined in Git (manifests, charts, configs)
2. A controller (ArgoCD, Flux) continuously reconciles cluster state to match Git
3. Changes to infrastructure or apps go through Git PRs

Why it matters

Audit trail (git log), easy rollback (git revert), declarative, observable drift.

Gotcha

Image tags in Git create CI/CD loops if not handled carefully. We use `[skip ci]` in bot commit messages to prevent.

Tip

Don't apply changes directly to the cluster (no `kubectl apply` from CI). Always go through Git. The controller will apply.

Interview line

“Adopted GitOps so every deploy is a Git commit reviewable in `git log`. Rollback is `git revert`. ArgoCD reconciles. No more ‘cluster diverged from Git silently’ surprises.”

Image promotion (build-once-deploy-many)

Plain English

Same image goes through dev → staging → prod. Don't rebuild per environment.

Technical definition

CI builds one image with a unique tag (typically git SHA). Promotion is changing references in different env config files (values.yaml) to point at that SHA. The bits are identical across environments.

Why it matters

The image you tested in dev is byte-for-byte the image in prod. Eliminates “works in staging” mystery.

Tip

Image promotion is a Git workflow:

1. CI builds image with sha-abc1234
2. PR updates values-dev.yaml to sha-abc1234
3. Merge → ArgoCD deploys to dev
4. After testing, PR updates values-prod.yaml to sha-abc1234
5. Merge → ArgoCD deploys to prod

Same image. Different env. No rebuild.

Interview line

“Build once, deploy many. Same image SHA flows through environments via values.yaml edits. The image tested in dev is the image in prod, byte for byte. Eliminates entire class of ‘works in staging’ bugs.”

Section K — Progressive Delivery

Blue/Green deployment

Plain English

Run two full versions side-by-side. Switch traffic over at once. Keep the old one running for fast rollback.

Technical definition

Active ReplicaSet (blue) serves traffic. New version (green) deployed alongside. Service selector flips from blue to green when ready. Old blue scaled down after a grace period.

Why it matters

Instant rollback — keep blue running for X seconds after promotion. If green has issues, flip back.

Gotcha

Costs 2x resources during deploy. For memory-heavy apps this can be significant.

Tip

For database migrations, blue/green is tricky. New version may need new schema; old version may not work with it. Use expand-contract pattern: make schema backward compatible, deploy green, run migrations gradually.

Interview line

“Blue/Green for stateless front-ends — fast rollback, easy reasoning. For backends with DB schema changes, expand-contract pattern: schema migrates first in a backward-compatible way, then code deploys, then old schema columns are dropped after the rollback window closes.”

Canary deployment

Plain English

Gradually shift X% of traffic to the new version. Watch metrics. If healthy, increase to 100%.

Technical definition

Route a percentage of traffic to the new ReplicaSet via Service mesh weights or Ingress. Steps: 10% → wait → 25% → wait → 50% → wait → 100%. Auto-abort on metric thresholds.

Why it matters

Catch issues that only manifest at scale with real users. Bug in 10% of traffic affects 10% of users, not 100%.

Tip

Pair canary with automated metric analysis. If error rate spikes during 10% phase, abort. Manual canary without metrics is just a slower outage.

Interview line

“Canary for backends where 10% of real traffic is statistically meaningful. Pair with metric analysis — error rate, p99 latency, business KPIs. Auto-abort on regression. Without metrics, canary just slows down failures.”

Argo Rollouts

Plain English

Kubernetes controller that does blue/green and canary deployments. CRD-based, integrates with ArgoCD.

Technical definition

CRD (Rollout) replacing Deployment. Manages ReplicaSets, Services, and pre-promotion analysis. CLI plugin `kubectl argo rollouts` for control.

Why it matters

First-class progressive delivery, not bolt-on. The blue/green and canary patterns are explicit in the CRD.

Tip

Common commands:

```
kubectl argo rollouts get rollout <name> -n <ns>
kubectl argo rollouts promote <name> -n <ns>
kubectl argo rollouts abort <name> -n <ns>
kubectl argo rollouts undo <name> -n <ns>
kubectl argo rollouts retry rollout <name> -n <ns>
```

Interview line

“Argo Rollouts for progressive delivery. CRD replaces Deployment. Supports blue/green and canary with built-in analysis hooks. Pairs naturally with ArgoCD — same project, same UX. We use it for frontend and backend in the lab.”

AnalysisTemplate

Plain English

“Check metrics before promoting.” Argo Rollouts hooks Prometheus into the rollout decision.

Technical definition

CRD defining queries (Prometheus, Datadog, Wavefront, etc.) and success/failure conditions. Used by Rollout as `prePromotionAnalysis` or `postPromotionAnalysis`.

Why it matters

Metric-gated promotion = automatic safety net. Don’t rely on humans watching dashboards.

Tip

Always have at least two metrics:

1. Min-traffic gate — verifies the new version is actually receiving traffic
2. Success-rate gate — verifies the traffic it receives succeeds

Without (1), (2) trivially passes on zero traffic.

Interview line

“Two-gate analysis: min-traffic (1 req/sec on preview) and success-rate (95% non-5xx). Both must pass 3 consecutive checks before promotion. Without min-traffic, success-rate trivially passes on zero traffic — false positive.”

Min-traffic gate

Plain English

“Don’t trust the success-rate gate if no traffic is flowing through the new version.”

Technical definition

Pre-promotion analysis metric that requires a minimum request rate on the preview Service before evaluating other metrics.

Why it matters

Without it, or `vector(1)` falsely returns 100% success on zero traffic. The promotion happens based on no evidence.

Tip

Drive traffic to the preview Service before promotion. Either:

- Have CI run synthetic smoke tests against `<svc>-preview`
- Use traffic replay tools (e.g., Diffy)
- Manual `kubect1 port-forward` and `curl` during analysis window

Interview line

“Min-traffic gate is mandatory. Without it, success-rate trivially passes on no traffic — false positive. We require 1 req/sec for 90 seconds before promotion. CI smoke tests drive that traffic.”

Section L — Observability

Metric

Plain English

A number measured over time. Request count. Latency. Memory usage.

Technical definition

Time-series data point: name, labels (dimensions), value, timestamp. Stored in a time-series database (Prometheus, Graphite, InfluxDB).

Why it matters

Metrics answer “what’s happening?” and “how often?”. Foundation of monitoring, SLO tracking, alerting, autoscaling.

Tip

Format like Prometheus exposition:

```
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",status="200"} 1234
```

Most modern apps either expose `/metrics` directly or have a Prometheus client library.

Interview line

“Apps expose `/metrics` in Prometheus format. Prometheus scrapes every 15s. PromQL queries aggregate across time and labels. For our lab the key metric is `istio_requests_total` from the istio-proxy sidecars.”

Cardinality

Plain English

How many unique combinations of labels a metric has. High cardinality = trouble.

Technical definition

Each unique combination of label values creates a separate time series in the database. Total cardinality = sum across all metrics. High cardinality consumes memory exponentially.

Why it matters

A typical Prometheus instance can handle millions of series. High-cardinality labels (user IDs, request IDs) blow this up. Prometheus OOM kills are usually a cardinality problem.

Gotcha

Don't use unbounded values as labels:

```
BAD: http_requests_total{user_id="abc123"}      ← unbounded
GOOD: http_requests_total{endpoint="/api/info"} ← finite set
```

Tip

Use histogram buckets, not raw values:

```
BAD: http_request_duration_seconds{value="0.234"}
GOOD: http_request_duration_seconds_bucket{le="0.25"}
```

Interview line

“Cardinality kills Prometheus. Never use unbounded labels — user_id, request_id, timestamps. Use buckets for histograms. We had to drop a label once because cardinality jumped from 10k to 10M and Prometheus OOMed.”

PromQL

Plain English

The query language for Prometheus. How you ask questions of the metrics.

Technical definition

Functional, declarative language for selecting and aggregating time-series. Supports rate calculations, percentiles, joins, and math operations.

Why it matters

PromQL is how you build dashboards, define alerts, and run AnalysisTemplates.

Tip

Common patterns:

```
# Request rate per second over 5-min window
sum(rate(http_requests_total[5m]))

# Group by status
sum by (status) (rate(http_requests_total[5m]))

# Success rate
sum(rate(http_requests_total{status!~"5.."}[2m]))
/
sum(rate(http_requests_total[2m]))

# 99th percentile latency
histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

Interview line

“PromQL: `rate()` for counters, `histogram_quantile()` for percentiles, `sum by ()` for aggregation. The [5m] window is the lookback. For analysis gates I use 2-minute windows — long enough to smooth noise, short enough to react.”

SLI / SLO / SLA

Plain English

- **SLI** — what you measure (success rate)
- **SLO** — what you aim for (99.9%)
- **SLA** — what you promise customers (refund if SLO breached for >1 month)

Technical definition

- **Service Level Indicator** — a specific metric measuring service quality
- **Service Level Objective** — internal target for that SLI, over a time window
- **Service Level Agreement** — external contract with consequences for missing the SLO

Why it matters

SLOs drive priorities. Error budget remaining? Ship features. Budget burned? Reliability work.

Tip

Start with one SLO per service. Pick the metric users care about most. Usually success rate of critical user journeys.

Interview line

“SLI = the metric. SLO = the internal target (99.9% over 28 days). SLA = customer-facing with consequences. SLO drives team priorities — if we have error budget, ship features. If burned, work on reliability. The SLA is the legal commitment.”

Error budget

Plain English

How much downtime you're allowed before missing your SLO. The “permitted failure” pool.

Technical definition

$(1 - \text{SLO}) \times \text{time}$. For 99.9% over 30 days, error budget = 43 minutes of permitted downtime.

Why it matters

Error budget = explicit license to take risks. Used it all in deploys? Slow down. Lots left? Ship more.

Tip

Burn rate alerting (Google SRE book): page on-call when budget burns too fast.

SLO: 99.9% over 30 days = 43 min budget

If we burned 50% in the first 2 days → fast burn → page

Interview line

“Error budget is permission to take risks. Burn rate alerts on fast consumption — multi-window, multi-burn-rate alerts (5min/1hr) per Google SRE book. The whole framework lets us have informed conversations about reliability vs feature velocity.”

Reading the Handbook — Pro Tips

1. **Don't read straight through.** Read one chapter, do the exercises, come back next day.
 2. **Type, don't copy.** Every command, every YAML — type it. Muscle memory matters.
 3. **Break things on purpose.** Best learning is recovery from mistakes you made.
 4. **Use the labs you've built.** This handbook is paired with a real working repo. Run it.
 5. **Talk through concepts.** Explain to a rubber duck. If you can't explain it, you don't understand it.
 6. **Take notes in your own words.** Don't highlight the handbook. Write your own summary.
 7. **Drill the interview questions.** Every chapter has them — actually rehearse out loud.
 8. **Compare to your previous knowledge.** “How is Kubernetes Service like a load balancer I already know?”
-

Common Cross-Cutting Tips

When you hit an error

1. **Read the error.** Slowly. Twice. 80% of errors tell you exactly what's wrong.
2. **Look at events.** `kubectl get events -A --sort-by='.lastTimestamp' | tail` shows recent issues.
3. **Check logs.** `kubectl logs <pod>` for app, `journalctl -u <service>` for systemd.
4. **Search the exact error message.** Verbatim, in quotes, in Google. Someone hit it before.
5. **Bisect.** Did it work yesterday? `git log` the last day's changes. Revert one at a time.

When learning a new tool

1. **Read the quickstart.** Just the quickstart. Resist the urge to read the whole docs.
2. **Build something small.** A Hello World, then your real use case.
3. **Read the source/issues.** Especially GitHub Issues — that's where real problems are documented.
4. **Find the mental model.** Every tool has one. ArgoCD = “sync Git to cluster”. Terraform = “diff state to cloud”.

When debugging Kubernetes

1. `kubectl get pods -A` - anything not Running?
2. `kubectl describe pod <name>` - events at the bottom?
3. `kubectl logs <name>` - what did the app say?
4. `kubectl logs <name> --previous` - if it crashed, last instance's logs
5. `kubectl exec -it <name> -- sh` - get a shell, poke around
6. `kubectl get events --sort-by='.lastTimestamp' | tail`
7. Try the same in a different namespace - namespace issue?
8. Try the same with a different image - image issue?
9. Try the same on a different cluster - cluster issue?

When writing code/YAML/HCL

1. **Format on save.** Use editor plugins. Manual formatting wastes time.
2. **Validate before commit.** `terraform validate`, `helm lint`, `kubectl apply --dry-run=client`.
3. **Commit small.** One logical change per commit.

4. **Write the test first** (or at least the test plan in the PR description).

Career tips

1. **Document everything.** Future you will thank you. Your team will too.
 2. **Automate the boring stuff.** If you do it 3 times, script it. 5 times, productize it.
 3. **Share your work.** Blog posts, talks, internal wikis. Visibility = career growth.
 4. **Learn one new thing every week.** Even 30 minutes of focused study compounds.
 5. **Be honest about gaps.** “I don’t know X, but here’s how I’d learn it” is a great answer.
-

End of Glossary